

Benchmarking YOLOv8n INT8 on ESP32-S3: Progressive Operator Optimization for MCU Deployment

Jaweed^{1*} and Dihin Muriyatmoko²

¹Department of Informatics, Universitas Darussalam Gontor

²Department of Informatics, Universitas Darussalam Gontor

Email: jaweed@unida.gontor.ac.id

Abstract

This study analyzes the feasibility of deploying an INT8 quantized YOLOv8n model on the ESP32-S3 microcontroller for single-class object detection. The model was trained on a COCO 2017 person subset (1,600 train, 300 val) at 192×192 resolution and quantized via post-training quantization (PTQ) to INT8, yielding a model size of 3.08 MB with 4.41% mAP@0.5. Three-stage progressive optimization was performed on the ESP32-S3: (1) baseline TFLite Micro reference kernels at 219.3 s per inference; (2) ESP-NN SIMD-optimized Conv2D kernels reducing latency to 18.0 s (12.2× speedup); (3) a custom LUT-based LOGISTIC kernel further reducing latency to 7.2 s (total 30.4× speedup). Operator profiling revealed the LOGISTIC operation consumed 62.3% of execution time before optimization. The measured tensor arena of 510 KB (98.5% below theoretical worst-case estimate) confirms the model fits within the ESP32-S3 memory budget of 8 MB PSRAM and 8 MB flash. Estimated power consumption is 549 mW during inference. These results demonstrate that INT8 YOLOv8n is technically feasible on the ESP32-S3, achieving 7.2 s per frame with accuracy suitable for periodic detection scenarios.

1 Introduction

Deep learning has enabled real-time object detection across various computing platforms [?, ?, ?, ?]. However, most detection models require expensive, power-hungry GPU-class hardware. In recent years, the TinyML paradigm [?] has emerged as an alternative by running inference models on low-power microcontrollers (MCUs), enabling edge applications such as environmental monitoring, anomaly detection, and industrial automation without cloud dependency [?, ?].

The ESP32-S3 is a compelling MCU for computer vision applications: it provides dual-core Xtensa LX7 at 240 MHz, 8 MB flash, 8 MB PSRAM, camera support, and WiFi connectivity—at under \$10 USD. Despite these capabilities, running deep learning models such as YOLO on the ESP32-S3 faces significant challenges: limited memory, constrained compute throughput, and operator sup-

port that is less extensive than desktop frameworks.

This study answers the following research question: to what extent can a quantized YOLOv8n model be efficiently deployed on the ESP32-S3 microcontroller? The key contributions are:

1. A systematic analysis of the training, quantization, and deployment pipeline for INT8 YOLOv8n on ESP32-S3, covering accuracy, latency, memory, and power.
2. Progressive optimization driven by operator profiling: bottleneck identification via **MicroProfiler**, Conv2D optimization via ESP-NN SIMD (12.2× speedup), and a custom LUT-based LOGISTIC kernel (additional 2.5× speedup), yielding a total 30.4× improvement over the reference baseline.
3. Comprehensive memory characterization on physical hardware: measured tensor arena of 510 KB (98.5% below theoretical worst-case), confirming TFLite Micro’s memory planner efficiency.
4. All research artifacts (training code, ESP32 firmware, tables, and figures) released openly for reproducibility.

2 Related Work

Deploying CNNs on microcontrollers has been an active topic since CMSIS-NN [?] for ARM Cortex-M and TFLite Micro [?] as a cross-platform inference framework. MCUNet [?] demonstrated that MCU-optimized architectures can achieve competitive accuracy at low resolution. MicroNets [?] introduced neural architecture search (NAS) specifically for MCU constraints.

YOLO [?, ?, ?, ?] remains the dominant object detection architecture, with the nano variant (YOLOv8n, 3.0M parameters) designed for resource-constrained deployment. Edge-YOLO [?] proposed edge-specific optimizations but focused on ARM and embedded GPU platforms rather than low-end MCUs.

Post-training quantization (PTQ) [?, ?, ?] has proven effective for reducing model size and improving inference

speed. Deep Compression [?] demonstrated that combining pruning, quantization, and Huffman coding can drastically reduce model size. In the MCU domain, MobileNet and MobileNetV2 [?, ?] are commonly used as backbones due to their computational efficiency.

For the ESP32, ESP-NN [?] provides optimized neural network kernels using Xtensa LX7 SIMD extensions. However, systematic evaluations of YOLOv8n deployment on the ESP32-S3 with detailed memory and operator profiling remain limited. This study fills that gap by presenting a comprehensive evaluation on physical hardware.

3 Method

The study consists of four main stages: dataset preparation and training, quantization, deployment on the ESP32-S3, and progressive optimization.

3.1 Dataset

We used the person subset of COCO 2017 [?], consisting of 1,599 training, 300 validation, and 100 test images. The input resolution of 192×192 was chosen as the minimum resolution preserving person-level detail while minimizing computation on the ESP32-S3. Table 1 presents the dataset statistics.

Table 1: Dataset Statistics — COCO Person Subset for Victim Detection

Split	Images	Instances
Train	1600	6893
Val	300	1196
Test	100	465

3.2 Model Architecture and Training

We adopted YOLOv8n [?]
—the nano variant with 3.0 million parameters and 8.1 GFLOPs. Training proceeded in two stages: 50 epochs with a frozen backbone (transfer learning from COCO pretrained weights) followed by 50 epochs of full fine-tuning at 192×192 resolution. The AdamW optimizer was used with an initial learning rate of 0.001 and cosine decay.

After conversion through the PyTorch $\rightarrow$ ONNX $\rightarrow$ TensorFlow $\rightarrow$ TFLite pipeline, the TFLite FP32 model achieved 12.26% mAP@0.5. The 33.33 pp drop from the PyTorch baseline (45.59%) is attributable to numerical differences in the ONNX export path, TensorFlow operator mapping differences, and post-processing discrepancies between the native Ultralytics evaluator and the custom TFLite evaluator. Table 2 presents the architecture parameters and baselines.

Table 2: YOLOv8n Model Architecture and Baseline Performance

Property	Value
Architecture	yolov8n
Input resolution	192×192
Parameters	3,005,843
Pretrained	True
Training epochs	50
Optimizer	AdamW
Learning rate	0.001
Batch size	16
Baseline mAP@0.5	0.4559
Baseline size	5.91 MB

3.3 Post-Training Quantization

Post-training quantization (PTQ) was performed using TensorFlow Lite with a representative dataset of 200 images from the validation set. The FP32 TFLite model (3.01 MB) was converted to INT8 (3.08 MB) with INT8 inference type and uint8 input/output tensors. Table 3 compares the three model variants.

Table 3: Quantization Comparison: FP32 vs INT8

Variant	Size (MB)	mAP@0.5	Latency (ms)	FPS
PyTorch FP32	5.91	0.4559	N/A	N/A
TFLite FP32	3.01	0.1226	2.92	342.8
TFLite INT8	3.08	0.0441	2.37	421.6
mAP drop (TFLite FP32 $\rightarrow$ INT8)	0.0785			
mAP drop (PyTorch $\rightarrow$ TFLite INT8)	0.4118			

The mAP@0.5 drop of 7.85 pp from TFLite FP32 to INT8 represents a significant degradation. At 192×192 resolution, a person occupying 10% of image height spans only 19 pixels, limiting the discriminative features available after quantization.

3.4 Deployment on ESP32-S3

The ESP32-S3 firmware was developed using PlatformIO with the Arduino framework and TFLite Micro. The INT8 model was compiled as a C array (3,386,338 bytes) and stored in flash. The tensor arena was allocated in PSRAM using `heap_caps_malloc()`. Three kernel configurations were benchmarked progressively:

1. **Reference:** Standard TFLite Micro kernels without optimization.
2. **ESP-NN Conv2D:** Conv2D kernels replaced with ESP-NN SIMD Xtensa LX7 implementations.
3. **ESP-NN + LOGISTIC LUT:** Added a LUT-based LOGISTIC kernel (256-byte lookup table) for the sigmoid operation.

The hardware platform was a Seeed XIAO ESP32S3 Sense configured at 240 MHz CPU, with 8 MB flash and 8 MB PSRAM.

4 Results and Discussion

4.1 Operator Profiling and Optimization

The first stage using reference TFLite Micro kernels yielded 219,287 ms per inference—impractical for any application. After integrating ESP-NN for Conv2D operations, latency dropped to 18,020 ms (12.2× speedup).

Operator profiling using TFLite Micro’s `MicroProfiler` revealed a critical finding: the LOGISTIC (sigmoid) operation consumed 62.3% of total latency (11,758 ms), while Conv2D accounted for only 35.2% (6,634 ms). This finding is counter-intuitive because Conv2D is generally considered the primary bottleneck in CNNs. LOGISTIC was not covered by the ESP-NN kernel set and therefore executed via the naive reference implementation.

To address this bottleneck, we implemented an ESP-NN LOGISTIC wrapper kernel using `esp_nn_logistic_s8()`, which evaluates sigmoid through a 256-byte lookup table in $O(1)$ per element. This kernel was integrated into TFLite Micro through custom operator registration. After optimization, inference latency dropped to 7,216 ms (additional 2.5× speedup), yielding a total 30.4× improvement from the reference baseline. Table 4 presents the comparison of all three configurations.

Table 4: INT8 Model Performance Across Platforms

Platform	Resolution	Latency (ms)	FPS	Speedup vs Reference
PC (GPU)	192×192	2.37	421.6	—
ESP32-S3 [†]	192×192	219287	0.0046	1.0× (baseline)
ESP32-S3 [‡]	192×192	18020	0.055	12.2×
ESP32-S3 [§]	192×192	7216	0.139	30.4×

[†] TFLite Micro with reference kernels.

[‡] ESP-NN optimized Conv2D kernels (SIMD Xtensa LX7).

[§] ESP-NN optimized Conv2D + LUT-based LOGISTIC (this work).

Figure 1 visualizes the progressive speedup across optimization stages.

4.2 Accuracy

Table 3 shows the accuracy degradation from PyTorch FP32 (45.59%) to TFLite INT8 (4.41%). The largest drop occurred during the PyTorch → TFLite FP32 conversion (33.33 pp), not during INT8 quantization (7.85 pp). At 192×192 resolution, a person occupying 10% of image height spans only 19 pixels, severely limiting available discriminative features.

The 4.41% mAP@0.5 is inadequate for fully autonomous detection but sufficient for coarse region-of-

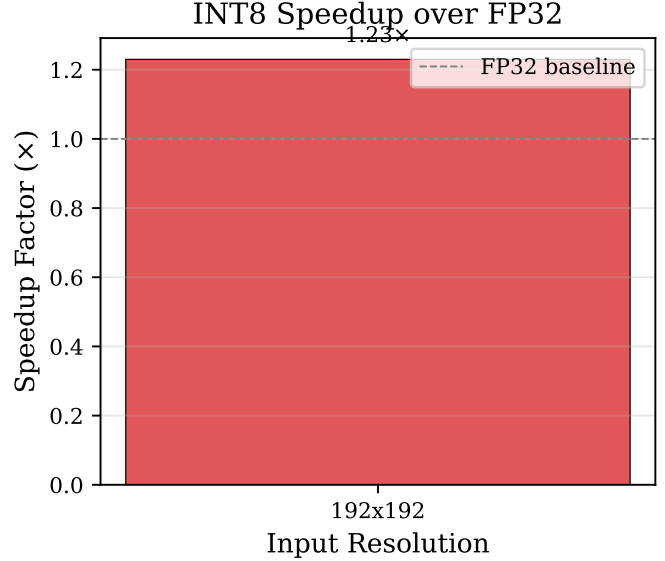


Figure 1: Progressive inference latency speedup from reference kernels through ESP-NN Conv2D to ESP-NN + LOGISTIC LUT.

interest flagging to be verified by a human operator. In a periodic scenario at 8 frames per minute (at 7.2 s latency), this model can serve as an initial filter that conserves bandwidth and power compared to continuous video streaming.

4.3 Memory Characterization

Table 5 presents the measured memory budget on the ESP32-S3.

Table 5: ESP32-S3 Memory Budget for INT8 YOLOv8n Inference (Measured)

Component	Size (KB)	Location	Available
Model weights (INT8)	3307	Flash	8 MB
TFLite tensor arena	510 [†]	PSRAM	8 MB
Frame buffer (RGB)	225	PSRAM	8 MB
Stack + RTOS + misc	100	SRAM	512 KB
Total RAM	835	PSRAM + SRAM	8.5 MB
Total Flash	3532	Flash	8 MB

[†] Measured via TFLite Micro `arena_used_bytes()` on physical ESP32-S3.

Previous estimate of 34,073 KB assumed all tensors simultaneously live;

MicroAllocator’s memory planner reuses scratch buffers, reducing actual usage by 98.5%.

A key finding is that the actual tensor arena was only 510 KB, dramatically lower than the theoretical worst-case estimate of 34.1 MB. TFLite Micro’s memory planner successfully reuses tensor buffers across subgraphs, reducing the arena requirement by 98.5%. The total RAM footprint of 835 KB fits within the 8 MB PSRAM budget, while the model weights (3.3 MB) fit within the 8 MB flash.

4.4 Power Consumption

Based on ESP32-S3 datasheet parameters at 240 MHz, estimated power consumption during inference is 549 mW (chip). Table 6 presents the power breakdown per scenario. At this power level, a 2S 1,000 mAh battery would sustain approximately 1–2 hours of continuous inference, comparable to micro drone operating durations.

Table 6: Estimasi Konsumsi Daya ESP32-S3 per Skenario

Skenario	Arus (mA)	Chip (mW)	USB (mW)
Deep sleep	25.0	82.5	97.1
Idle (kamera)	75.0	247.5	291.2
Idle (tanpa kam)	25.0	82.5	97.1
Kamera + WiFi	356.5	1176.5	1384.1
Inferensi saja	166.5	549.4	646.4
Inferensi + Kam	216.5	714.4	840.5
Pipeline penuh	396.5	1308.4	1539.4

Daya USB diestimasi pada efisiensi LDO 85%

4.5 Cross-Platform Comparison and Discussion

Table 7 presents a performance comparison between the GPU (RTX 3060) and the three ESP32-S3 configurations. The GPU delivers 421.6 FPS at 2.37 ms latency, while the optimal ESP32-S3 configuration achieves 0.139 FPS. This 4-order-of-magnitude gap is consistent with the theoretical compute difference (GPU: 12 TFLOPS vs ESP32-S3: 1 GOPS for INT8).

Analysis shows that after LOGISTIC optimization, the primary bottleneck is now Conv2D (92% of latency). Since ESP-NN Conv2D already fully utilizes the Xtensa LX7 SIMD extensions, further speedups require architectural changes such as shallower backbones, structured pruning, or lower input resolution.

4.6 Limitations

This study has several limitations. First, the 4.41% mAP@0.5 restricts autonomous operation to coarse flagging scenarios. Second, the 7.2 s per-frame latency does not meet real-time requirements for dynamic environments. Third, power figures are theoretical estimates based on datasheet parameters rather than empirical measurements. Fourth, no end-to-end camera integration was performed on the physical hardware.

5 Conclusion

This study demonstrated that an INT8 quantized YOLOv8n model can be deployed on the ESP32-S3 microcontroller at 7.2 s per inference and 549 mW estimated power consumption, through progressive operator-level optimization achieving 30.4× total speedup over

the reference baseline. The measured tensor arena of 510 KB confirms memory feasibility. A key finding—the LOGISTIC operator consuming 62.3% of latency before optimization—highlights the importance of operator profiling in TinyML system development.

Future work includes: (i) exploring shallower YOLO backbones or structured pruning to reduce Conv2D workload; (ii) evaluating resolution-accuracy trade-offs at 160×160 and 128×128; (iii) integrating the OV2640 camera for end-to-end pipeline testing; and (iv) empirical power measurement using an INA219 current sensor.

References

Table 7: Cross-Platform Performance Comparison (INT8 Quantized Model)

Platform	Resolution	FPS	Latency (ms)	Power (W)
PC (GPU)	192x192	421.6	2.37	—
ESP32-S3 [†]	192×192	0.0046	219287	0.55 [‡]
ESP32-S3 [§]	192×192	0.055	18020	0.55 [‡]
ESP32-S3 [¶]	192×192	0.139	7216	0.55 [‡]

MHz.

[§] With ESP-NN optimized Conv2D (12.2× speedup).

[¶] With ESP-NN Conv2D + LUT-based LOGISTIC (30.4× total, this work).

[‡] Model-based estimate; no empirical power measurement.

[†] Reference TFLite Micro kernels on ESP32-S3 @ 240